

Luis Madrigal

1. The first article read was "Non-linear system modeling using LSTM neural networks" by Jesús Gonzalez and Wen Yu. In this paper, they developed an LSTM model for non-linear modeling; this is important because LSTMs tend to struggle with this type of data. To do this, they used backpropagation through time methods. Their models were able to produce better results than existing neural models when applied to two non-linear system examples.

2. The second article read was "Unsupervised Anomaly Detection With LSTM Neural Networks" by Tolga Ergen and Suleyman Serdar Kozat. In this paper, they aim to develop an unsupervised method for detecting anomalies in variable-length data sequences. This is of great importance since traditional methods tend to struggle with time-series dependencies and varying sequence lengths. The paper achieves this by combining STM networks with OC-SVM and SVDD algorithms and trains them using gradient-based and quadratic-programming methods. Testing on digit, occupancy, financial, and network datasets showed higher AUC scores when compared to traditional methods. However, some parameter tuning was required.

1. The first article read was "Ensemble Method for System Failure Detection Using Large-Scale Telemetry Data" by Priyanka Mudgal and Rita Wouhaybi. In this paper, the authors seek to detect PC system failures using large-scale telemetry data; this is of great importance since failures disrupt users and can lead to financial losses. The data is composed of CPU, memory, disk, temperature, and system data. This data is preprocessed, and then an LSTM encoder is combined with Isolation Forest, OCSVM, or LOF. LSTM with Isolation Forest performed best, as it was able to achieve 87.38% accuracy with the lowest inference time. The dataset was very imbalanced; this caused poor detection results in abnormal cases.

2. The second article was "AI-DRIVEN ROOT CAUSE ANALYSIS FRAMEWORK FOR DISTRIBUTED MICROSERVICES ARCHITECTURES" by Awodele S. O.; Faruna, J. O.; Mustapha M. M.; Ojuawo O. O.; Olorunyomi O. B.; Chukwulobe I.; Fayemi T. A. In the paper, "RCASage" is proposed as an AI-based framework designed to identify the causes of failures in distributed microservices. This is important because manually troubleshooting these services can be costly in time and resources. "RCASage" is a combination of telemetry data, LSTM anomaly detection, NLP, dependency graphs, graph neural networks, causal inference, and AI. The paper mentions related studies that suggest that a framework such as "RCASage" can reduce resolution time by over 90%, improving also diagnostic accuracy. The paper works more as a conceptual design of such a framework; however, the paper acknowledges that the cold start, scalability, and real-world validation could be limitations.

Duc Nguyen

Paper 1: <https://microsoft.github.io/OpenRCA/Links> to an external site.

This paper introduces OpenRCA, a public benchmark and evaluation framework for testing whether large language models can identify the root cause of software failures from operational telemetry. It contains 335 real failure cases drawn from three enterprise systems, paired with more than 68 GB of de-identified logs, metrics, and traces.

Results: Claude 3.5 tops the table at 11.34% with RCA-agent, falling to 5.37% under oracle sampling and 3.88% under balanced sampling. Proprietary models beat open-source ones consistently, and RCA-agent beats both sampling strategies.

Paper limitation: all failures come from distributed systems, with monolithic architectures left to future work; the queries are synthesized rather than real failure reports, since privacy concerns blocked access to first-hand reports; and RCA-agent's scalability comes at the cost of demanding stronger error tolerance from the model.

Paper 2: <https://www.mdpi.com/2073-8994/14/3/454?utmLinks> to an external site.

LogLS detects anomalies in system logs by treating the log as a natural-language sequence and modeling it from two directions at once, the preorder relationship (what came before an event) and the postorder relationship (what comes after). Two LSTMs, one per direction, are combined into a single detector. The authors position it explicitly as an optimization of DeepLog aimed at solving the poor prediction performance of LSTM on long sequences. A secondary goal is handling log patterns the training set never saw, via a feedback/update mechanism.

One limitation from the paper: the model still cannot predict log execution paths that never appear, and they list this as future work along with extending detection to log parameters rather than just execution mode.

Article 1: <https://arxiv.org/html/2407.00048v1>Links to an external site.

This paper identifies system anomalies on end user's computers by collecting data in the form of telemetry, where an LSTM autoencoder is used as a feature encoder to feed three different classical anomaly detectors including isolation forest, one class support vector machine, and local outlier factor algorithm. An LSTM encoder is utilized for encoding the collected telemetry data, which are then processed by the classical model to identify any outliers.

Architecture. A two level LSTM encoder in an autoencoder, which minimizes reconstruction loss, that generates a fixed-range feature vector Y . Timestep is equal to 1. Y is further inputted into isolation forest, OCSVM, and LOF separately. Training: learning rate 0.001, batch size 16, tanh activation, Huber loss function, Adam, 25 epochs; contamination 0.01 for isolation forest and LOF, ν 0.01 for OCSVM.

Limitation:

Speed gains, not accuracy gains. The honest contribution is 1.5× faster training and inference at comparable accuracy, not better detection.

Aggressive information loss in preprocessing. Collapsing a day of 5-second samples into one of three categorical levels discards nearly all temporal structure — which sits oddly with using an LSTM to capture temporal correlation. Setting timestep to 1 compounds this; the LSTM is barely operating as a sequence model.

Article 2: <https://arxiv.org/pdf/1503.04069>Links to an external site.

Ablation Study of the LSTM Architecture Itself. In this study, the authors use the basic LSTM as a starting point and then perform an ablation study of 8 variants of this architecture, with each variant having only one difference from the starting point. The study was performed on three different domains: speech recognition, handwriting recognition, and polyphonic music modeling.

Scale: 5,400 experiments, equivalent to 15 years of CPU time, making it the largest such experiment ever done on LSTMs.

Cameron Ball

The first article, OpenRCA: Can Large Language Models Locate the Root Cause of Software Failures?, describes the OpenRCA dataset that I am using for my project. The dataset contains 335 software failure cases across three enterprise systems and includes metrics, logs, and traces. The paper was helpful because it gave me a better understanding of the dataset's structure and how the different types of telemetry are connected to software failures. It also helped me understand some of the challenges involved with working with multiple sources of telemetry.

The second article, Systematic Evaluation of Deep Learning Models for Log-based Failure Prediction, evaluated different deep learning approaches for predicting software failures from system logs. The paper was useful because it focuses specifically on failure prediction and shows how different approaches can perform depending on how the log data is represented and processed. This relates to my project because I will be transforming the OpenRCA telemetry into sequential data before using it with my LSTM model.

The first article, LogLS: Research on System Log Anomaly Detection Method Based on Dual LSTM, focused on using LSTM networks to detect anomalies in system logs. The paper was useful for understanding how sequential log data can be processed with LSTM models and how the structure and length of the input sequences can affect detection performance. This relates closely to my project since I will also be using sequential telemetry data to predict software failures.

The second article, Software failure time series prediction with RBF, GRNN, and LSTM neural networks, focused specifically on predicting software failures using different neural network approaches. The LSTM model performed well compared with the other approaches, which supports my decision to use LSTM as the primary model for my project. This article was especially relevant because it applied machine learning directly to software failure prediction rather than just general anomaly detection.